

claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_4849400f-9d3\agent-a7ed4abfc8764649f.jsonl`
- SHA-256: `5eb2abe679a32dca65e0d9c59482b92ed1d9d0ce09aa7c7b7540ed50ae63ac1d`
- Source modified: `2026-06-21T09:27:15+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_4849400f-9d3`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`
```

Transcript

1. user (2026-06-21T09:24:52.596Z)

You are mapping OWNER-GATED decisions for the badminton-highlight-indexer ENGINE serving track (project doc docs/COMPUTE_DECOUPLED_SERVING/README.md). Context: M0-M4 are SHIPPED (the config/input/output/device SEAM layer: deployment.profile+presets, input_backend, output base, DeviceContext). NOW the team is on M5+. The owner wants to BATCH-DECIDE every gate so the maximum independent code path is unblocked. The discipline: ONE isolated PR per milestone off origin/master, default-OFF + golden/cross-OS-gated for any core-caller change, mocked tests for anything not runnable locally (no GPU/cloud here). KEY DISTINCTION you must draw for every gate: separate (a) CODE that CAN be written + merged NOW behind default-OFF with mocked/faked tests (no owner needed) from (b) the precise OWNER PREREQUISITE (real GPU run, GCP spend, counsel/consent review, real prices, OAuth creds, a design pick). For each gate return the structured fields. Ground every claim in real file:line or doc anchors ? read the code, do not guess. Be exhaustive within your area but do not stray outside it.

YOUR AREA: code-reality for the CLOUD-SERVING compute substrate (M6/M7) ? can the dispatch shim + container image be WRITTEN and mock-tested now, deferring only the GCP spend/run?
READ: backend/worker/__main__.py (cmd_infer/cmd_train, the profile-agnostic worker), backend/api/job_worker.py + backend/api/uploads.py (async job control plane, chunked upload), backend/infrastructure/database.py (jobs table, claim_due_job CAS, ExecutionPolicy
WAIT_AND_RESUME / JobWaiting, migration ladder PRAGMA user_version), backend/storage/__init__.py + backend/storage/*.py (StorageBackend ABC, parse_uri, gcs backend), and backend/pipeline/segmenters/trajectory_hybrid.py::_resolve_runner (the compute seam).
Determine concretely: what already exists for Cloud Run dispatch (the job table, WAIT_AND_RESUME reschedule, worker CLI fetch/put), what is NET-NEW (a dispatch shim that enqueues a Cloud Run job + a Dockerfile with ffmpeg+CUDA+WASB+fonts), and whether that net-new code can ship behind default-OFF with MOCKED dispatch tests (no real Cloud Run). Frame the gate as "owner approves writing the code now + defers spend" vs "hold entirely". Report M6 and M7 as gates.

2. assistant (2026-06-21T09:24:55.310Z)

I'll map the M6/M7 cloud-serving compute substrate gates. Let me start by reading the project doc and the core files in my area.

3. user (2026-06-21T09:24:58.558Z)

Your configuration specifies to merge with the ref 'refs/heads/feat/serving-m4-device-context' from the remote, but no such ref was fetched.

4. user (2026-06-21T09:24:58.961Z)

```
1      # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2
3      > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** .
**Owner:** Avi . **Created:** 2026-06-21 . **Last updated:** 2026-06-21
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to
```

```

`docs/archives/past_projects/` when DONE)
5      > **Tracking anchors:** $7 progress tracker is the source of truth; pointer in memory
`current-state` + `docs/NEXT_STEPS.md` once work starts.
6      > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) $3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7      > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
**compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an
afterthought.
8      > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10     ---
11
12     > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video
? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next
to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13,
a headline feature), and execution is **never a surprise** ? auto-detect **suggests**, the user
**confirms** (D15).
13
14     ## 0. TL;DR
15     The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three
deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch
on**. A **`ComputeBackend` (deployment profile)** decides **where** the core runs (a local process
vs a Cloud Run GPU job); a **`StorageBackend`** decides **where bytes come from** (local FS / USB
/ mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local**
(local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive /
bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus
**cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the
same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the
**parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.
16
17     ## 1. Problem & goal
18     - **Why now:** DECOUPLED_COMPUTE proved compute **portability** (M0?M2 + M3.5 on a GCP L4)
and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero +
cost guard)** become this project. The owner needs **both** a hosted service **and** a truly-local
mode, with the **core (detect+stitch) unaffected** by which one runs.
19     - **The two user-facing outcomes** (owner, 2026-06-21):
20     - **a) Hosted service** ? operate on small **local uploads (`<2GB`)** **and** load from
**gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching**
(stitching is compute-intensive ? run it where its cost is accounted for).
21     - **b) Truly-local** ? on a local machine with videos on **local drives/USB + a local
GPU**, run inference **and** stitching **completely locally**.
22     - **"Good" looks like:** the same `video ? rallies ? reel` core gives identical windows
+ stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup
choice**, never a code branch in the core.
23
24     ## 2. Decisions locked
25
26     | # | Decision | Source | Implication |
27     |---|-----|-----|-----|
28     | D1 | The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile
branch inside it**. | design `[verified]` | All profile difference lives in config + the two
registries. |
29     | D2 | **Two registries:** `ComputeBackend` (where) selected by `deployment.profile` +
`compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`. | ADR-014
| Realizes PLATFORM_ARCHITECTURE's `local`/`hosted`/`byo_worker`. |
30     | D3 | **Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI);
`byo_worker` reserved/deferred. | ADR-014 | Enum/seam reserved so BYO isn't precluded. |
31     | D4 | In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so
CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would
hide real cost. |
32     | D5 | **Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory);

```

****detect**** is byte-exact only same-GPU. | deploy report 2026-06-20 | Cross-GPU asserted at the rally/window level, never CSV bytes. |

33 | D6 | ****Default-OFF, smallest-safe-first, one PR per milestone****; any core-caller change (e.g. configurable output base) ships behind a flag + a Launch Report. | **[[launch-report-convention]]** | Zero-regression discipline. |

34 | D7 | ****Cloud Run cold-start: SCALE-TO-ZERO**** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). ****No warm pool**** (that ~\$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane only on a UX complaint. |

35 | D8 | ****Pricebook = a versioned DB table****; cost computed in ****USD**** (matches GCP billing = one source of truth), ****displayed in INR**** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change. | owner 2026-06-21 | M8. |

36 | D9 | ****Edge auth = Cloudflare Access**** (reuse the site's existing CF). Lock the Cloud Run ****ingress to the CF proxy**** + ****verify the `Cf-Access` JWT signature**** (so a direct hit to the Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity system. |

37 | D10 | ****Free-tier "data-for-reels" trade:**** free reels in exchange for ****training rights**** (uploaded footage + derived trajectories/labels); ****paid tier = no-train**** (privacy is the paid feature). ****Carve-outs:**** train ONLY on attested ****ADULT**** footage (minor footage ? reel yes, training pool ****NO**** ? DPDP/GDPR need verifiable parental consent); train on ****DERIVED**** data + ****auto-delete raw****; ****ToS counsel-reviewed****; live training-on-uploads ****gated to M9**** (public signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties **[[paper-coauthor-aim]]** / PERSONALIZATION flywheel. |

38 | D11 | ****Quality floor: Gemini handoff ON**** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a ****Launch Report****; the owned model runs in ****shadow/eval**** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |

39 | D12 | ****Data-flywheel harness is IN SCOPE (owner add, Q5):**** capture the (consented, adult) uploaded video + the Gemini reel/rally output ? ****reliably store**** in the per-video workspace/bucket ? ****run shadow evals**** (owned-vs-Gemini, dual-eval-set) ? ****promote good ones to the golden TRAINING set**** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). | owner 2026-06-21 | new ****M11****; ties D10 (consent) + **`data\_pipeline/`** + the eval/experiment harness. |

40 | D13 | ****Profile is user-switchable ANYTIME ? a headline FEATURE.**** The same user runs ****local today, cloud tomorrow, back to local**** ? it's just a config/startup choice; the pure core gives ****equivalent**** output either way. ****Advertise it**** ("your machine *or* our cloud ? your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the ****rally/window**** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is ***between*** jobs. |

41 | D14 | ****NORTHSTAR ? operate toward this:**** a ****mobile, app-like**** flow ? point at a ****Google Drive**** video ? ****fully cloud-native**** run ? the stitched, ****ready-to-share**** reel lands ****back in Drive, next to the source****. | owner 2026-06-21 | Implies a ****`gdrive-api` (OAuth) StorageBackend**** (read source + write reel back) ? distinct from the local ****mount**** backend; ****resolves S8 #7****, new ****M12****. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the **`StorageBackend`** ABC accommodates it). |

42 | D15 | ****Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.**** A GPU owner may still choose cloud; ****cloud (= spend) is never entered silently****; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends S4.3 (auto-detect = a proposed default, not an auto-decision). |

43

44 **## 3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end***

45 This project is the latest step in a long decision chain; each ADR contributed a piece and a ****subtlety that carries forward****:

46

47 | ADR / doc | Contributed | Subtlety carried into this project |

48 |---|---|---|

49 | ****ADR-005**** | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served model + the ****container image**** (ffmpeg, fonts, weights) stay commercial-clean. |

50 | ****ADR-008**** | Owned-model topology; ****train==serve** featurizer single-sourcing** | The "core unaffected across profiles" is the same parity discipline, extended from train/serve to ****local/cloud****. |

51 | ****ADR-009**** | KhelSutra Guru; ****local-first delivery**** | The ****truly-local profile**** is the direct continuation ? privacy/offline self-host stays first-class. |

```

52      | **ADR-010** | DECOUPLED: D0?GCP pivot | The cloud-compute origin (GPU + co-located
bucket). |
53      | **ADR-011** | DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4
(billing/cost-guard)**; override the "rent nothing / not a service" posture | **This project
realizes M3/M4.** Carries $06's owner-gated gates: **DPA/consent for non-owner uploads,
collector `md5` keying, WASB-C3**. |
54      | **ADR-012** | GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving
model**; **NVDEC** named as the decode lever | The serving substrate + the 7-step seed scope
this expands. |
55      | **ADR-013** | Drop D0; **GCP sole compute stack**; $200 D0 credits ? non-compute only
| The provider for cloud-serving; D0 Spaces remains only a possible S3 *storage* target. |
56      | **? ADR-014 (new)** | **Compute-decoupled serving:** one pure core, profile-selected
compute+storage; stitch-where-metered | This doc. **Refines** ADR-011/012, does **not**
supersede them. |
57      | PLATFORM_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The
abstraction this realizes in code. |
58      | VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md | L0 fully-local tier / `LocalDesktop`
backend | The truly-local profile = these, productionized. |
59      | 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB
hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes. |
60      | DEPLOY_REPORT_GCP_2026-06-20 (archived) | First real L4 run; **cross-GPU sub-pixel
parity finding** | Why D5 (parity at window level, not CSV bytes). |
61
62      ## 4. Design / architecture
63      See [ `architecture.svg` ](architecture.svg) for the picture. **The core never changes;
the box around it does.**
64
65      ### 4.1 The core contract (must stay byte/behaviour-identical) `[verified]`
66      1. **DETECT (GPU):** `DetectorRunner.run_predict(video_path, output_dir, video_id) ?`
CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable
impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ?
emit the **identical CSV schema**. The trajectory CSV is the stable artifact boundary.
67      2. **WINDOW (CPU pure fn):** `trajectory_to_action_windows(points, fps, frame_width, ?)`
? windows` (`tracknet_runner.py:281`). A shared `@staticmethod`, zero GPU/IO/randomness ? same
trajectory + config = byte-identical candidate windows. Used by every runner *and* the
eval/regression stack verbatim.
68      3. **STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments,`
out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffmpeg ? watermark
? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.
69
70      **Non-goal (the trap to avoid):** **no* code branch inside detect/window/stitch keyed on
profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-
identical to CONTROL).
71
72      ### 4.2 The deployment profiles
73      | Profile | Compute | Storage | Orchestration | When |
74      |---|---|---|---|---|
75      | **truly-local** | `compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto`
(WSL); stitch **in-process** | `local` (or `gdrive` = mounted Drive) | `python -m backend.main`
or `worker infer` CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-
box ? owner (b) |
76      | **cloud-serving** | `compute_target=cloud_run`; `detector_impl=native` (L4, cuda);
**detect AND stitch on the same Cloud Run job** | `gcs` (or gdrive mount / assembled upload);
per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job
pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero | Hosted
SaaS ? owner (a) |
77      | **cpu-mock** | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) |
`local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free
dev |
78      | *byo_worker* (deferred) | tenant GPU via outbound pull agent | tenant's own bucket
(signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ?
**DEFERRED**, enum/seam reserved |
79
80      ### 4.3 Profile selection (the "startup/setup config") `[design]`
81      ONE new top-level `deployment` section on `AppConfig` + preset application + env auto-

```

```

detect in `load_config`:
82     - `deployment.profile ? {truly-local | cloud-serving | cpu-mock}` (empty = today's
manual knobs, **fully backward-compatible**).
83     - Selecting a profile applies a **coherent preset bundle** of *existing* knobs that
today drift independently: **INPUT** (new `input_backend`/`input_root`, parallel to the output-
side `StorageConfig`), **COMPUTE** (new `compute_target` enum locking `detector_impl` +
`skip_ai_handoff` + workspace strategy), **STORAGE**
(`storage.backend`/`workspace_root`/`single_folder_per_video`, all already exist).
84     - **Precedence:** built-in defaults ? profile preset ? `config.json` ? env
(`RALLY_DETECTOR_IMPL`, `WASB_*`, `DEPLOYMENT_PROFILE`, ?) ? worker `--set k=v`. Every knob
stays individually overridable.
85     - **Env auto-detect SUGGESTS, the user CONFIRMS (D15 ? no surprise execution):** the env
gives a *proposed* default (`K_SERVICE` ? cloud-serving; `CI=true` ? cpu-mock; else
`torch.cuda.is_available()` ? truly-local), but the user **explicitly confirms or overrides** it
? a GPU owner may still pick cloud, and **cloud (= spend) is NEVER entered silently**. The
active profile is always surfaced (logged/shown). **Per-profile startup checks fail/warn fast**
(cloud-serving without GCS creds/GPU ? loud error before any job).
86     - The worker (`cmd_infer`/`cmd_train`) is **already profile-agnostic** (reads
`config.storage`, accepts `--config`/`--set`) ? no worker rewrite.
87
88     ### 4.4 Compute placement ? and *why stitch runs on Cloud Run* `[design]`
89     DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In
**cloud-serving, stitch is co-located in the Cloud Run job** so `gpu_active_seconds` (detect) +
`wall_seconds` (detect+stitch) + `bytes_out` (reel egress) land on **one metered invocation** ?
the per-job cost ledger. Running stitch on a laptop would leave that cost
**invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in
the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get
bursty ? the job table + `claim_due_job` already supports both.)
90
91     ### 4.5 Reuse map ? most of the seams already exist `[verified]`
92     **Exists:** the 3-stage core; `_resolve_runner` (the single compute seam);
`StubDetectorRunner` (`replay_csv` = byte-exact $0 anchor); `StorageBackend` ABC + 4 backends +
`StorageRef.parse_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control
plane (`jobs` table, `claim_due_job` CAS); `ExecutionPolicy WAIT_AND_RESUME`/`JobWaiting`
(carries to Cloud Run dispatch verbatim); the chunked-upload router (<2GB); `skip_ai_handoff`
(LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video
workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user_id` columns
exist). **Net-new:** the `deployment` section; `input_backend`; configurable output base (kill
hardcoded `output`/); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-
auth header extraction; startup env detection.
93
94     ### 4.6 The data-flywheel harness (D12 / M11) `[design]`
95     Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip
Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline
additionally:
96     1. **Captures** the source upload + the Gemini reel + the rally output
(intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-
consent auto-delete path.
97     2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw
video kept only for the training pool, not the no-consent path).
98     3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini
(`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks
the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
99     4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING
rows** (human-reviewed labels via the annotation flow ? `data_pipeline`/), growing the corpus ?
the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.
100     This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the
PERSONALIZATION contribution model): every consented free reel makes the owned model better.
Reuses `data_pipeline`/, `backend/eval`/, and the M9 consent gate; **adults-only**, raw-
retention gated by the D10 consent.
101
102     ## 6. Honest limits ? what this does NOT do / prove
103     - A green CI suite proves **plumbing + determinism (window+stitch) + profile-parity**
for $0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-
side, real-video).

```

104 - ****Cross-GPU detection is NOT byte-identical**** ? parity is a window-level claim, not a CSV-byte claim.

105 - The cloud `0.1.0-l4` weights are ****n=2 plumbing-grade**** ? cloud-serving ships with ****Gemini handoff ON**** until the owned model clears a floor (open Q).

106 - This does not resolve ****WASB-C3**** (sharing a trained model) or the ****DPA/consent**** gate ? both remain owner/counsel-gated before public, non-owner serving.

107

108 **## 7. Deliverables & progress tracker** ? ****source of truth****

109 Legend: ? Todo . ? In progress . ? Done . ? Gated. ****One small PR per row****, off `origin/master`, no stacking. Default-OFF where it touches core callers.

110

ID	Deliverable	Depends	Gated?	Status	PR
M0	This design doc + project dir + `runlogs/` + ADR-014			?	owner sign-off
M1	`deployment.profile` + `compute_target` config + preset bundles + precedence/auto-detect; unit tests (no behaviour change, profile=' default)			?	
M2	`input_backend`/`input_root`; wire main CLI/API input through `StorageRef.parse_uri` (local=identity); storage-fake tests			?	
M3	Configurable output/scratch base ? kill hardcoded `output/` (`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`** ; golden + stub-E2E byte-identical			?	
M4	`DeviceContext` + WASB **CPU-fallback** ; unified healthcheck/device wiring			?	
M5	**truly-local profile** end-to-end (preset + startup checks); first committed **runlog** (truly-local sample run)			?	
M6	Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) + **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests			?	
M7	**cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet)			?	
M8	Per-job **cost ledger** (v_migration: `owner_id, gpu_active_seconds, wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost`			?	
M9	**Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate wiring			?	
M10	`byo_worker` / zero-infra-BYO ? **design-only, DEFERRED**			?	
M11	**Data-flywheel harness (D12):** on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9).			?	
M12	**gdrive-api` (OAuth) StorageBackend (D14 Northstar):**</b> cloud reads the source from + writes the stitched reel <b>**back to the user's Google Drive next to the source**</b> (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC.			?	

127 ****Testing strategy is a first-class deliverable**** ?
[`TESTING\_STRATEGY.md`](TESTING_STRATEGY.md). ****Run logs / sample runs for posterity**** ?
[`runlogs/`](runlogs/).

128

129 **## 8. Open questions** ? owner *(blocking-gates vs nice-to-knows)*

130 ****?** The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:

131 1. ? ****Cold-start ? D7**** (scale-to-zero, accept cold-start).

132 3. ? ****Pricebook ? D8**** (versioned DB table; USD internal / INR display).

133 4. ? ****Edge auth ? D9**** (Cloudflare Access; verify Cf-Access JWT, lock ingress to proxy).

134 5. ? ****DPA/consent ? D10**** (free-tier data-for-reels trade; adults-only; derived-data;

counsel-gated to M9).

135 6. ? **Quality floor** ? D11** + the **data-flywheel harness** D12 / M11** (Gemini-ON until the owned model clears the floor; capture?store?eval?goldenize meanwhile).

136

137 **Still open (non-gating, smaller):**

138 2. **Stitch placement:** always co-located in the detect job (simplest, one metered unit) vs split to a cheap CPU Cloud Run service when bursty? **Default** co-located unless metrics say otherwise.*

139 7. ? **RESOLVED** ? D14:** cloud needs a **gdrive-api** (OAuth) **StorageBackend** (read the source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ? distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.

140 8. **NVDEC/NVENC** in the Cloud Run image (cut stitch wall-time/cost) vs libx264 (determinism/parity)?

141 9. **Input cap:** is **<2GB** a hard client reject, or accept to the 4GB upload cap and route larger files elsewhere?

142 10. **Directory name:** **COMPUTE_DECOUPLED_SERVING/** (chosen) vs keep **CLOUD_RUN_GPU_SERVING/** ? confirm.

143

144 **## 9. Definition of done**

145 - [] M1?M4 land (config + input + output-base + device layer), each green on golden cross-OS + experiment no-regression.

146 - [] **Profile-parity test green** ? truly-local vs cloud-mock produce byte-identical windows + segment ranges.

147 - [] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog** / **DEPLOY_REPORT** with a real sample run.

148 - [] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth (M9) gates multi-tenant.

149 - [] **Data-flywheel harness** (M11) captures?stores?shadow-evals?goldenizes consented adult jobs; the owned model is on a measured path to the D11 floor.

150 - [] ? flip Status DONE, archive to **docs/archives/past_projects/**.

151

152 **## 10. References**

153 **ADR lineage:** ADR-005/008/009/010/011/012/013 ? **ADR-014** (**docs/archives/decisions/DECISIONS.md**). **Strategy:** **PLATFORM_ARCHITECTURE.md**, **HOSTING_PLAN.md**, **VIDEO_LOCALITY_MODEL.md**, **DESKTOP_APP_PLAN.md**, **LOCAL_APPLIANCE_ARCHITECTURE.md** (Khelsutra). **Archived predecessor:** **docs/archives/past_projects/DECOUPLED_COMPUTE/** (esp. **05-COST-ATTRIBUTION.md**, **07-COMPUTE-ABSTRACTION.md**, **06-RISKS-AND-OPEN-QUESTIONS.md**, **DEPLOY_REPORT_GCP_2026-06-20.md**). **Code anchors** **[verified]:** **backend/pipeline/detectors/base.py:164**, **tracknet_runner.py:281**, **stitcher/compiler.py:303**, **segmenters/trajectory_hybrid.py:65,237**, **backend/storage/***, **backend/worker/__main__.py:88**, **backend/api/{uploads,job_worker}.py**, **backend/infrastructure/database.py**. **Design provenance:** workflow **wf_56274ff0** (4 code-mappers ? synthesis; 58 findings).

154

155 ---

156 **### Changelog**

157 - 2026-06-21 ? created from the design workflow output; M0 in progress.

158 - 2026-06-21 ? **M0 APPROVED**; **M1 landed** ([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)): **deployment.profile** + **compute_target** + preset bundles + precedence/auto-detect-as-suggestion (D15) on **AppConfig**. Pure/additive, **profile=****'** default == today's behaviour (zero core change); presets touch only existing knobs (no M2 **input_backend** / M3 output-base over-reach). Suite 1068 green, cov 84.11%.

159 - 2026-06-21 ? **M2 landed** ([#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280)): **input_backend**/**input_root** + URI-aware main API/CLI input via **resolve_input_uri**/**acquire_local_input** (local = identity passthrough, byte-for-byte unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; **local://** accepted on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report provenance. Suite 1105 green, cov 84.84%.

160 - 2026-06-21 ? **M3 landed** ([#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282)): configurable output/scratch base via **backend/utils/paths.output_base(config)** ? the 3 segmenters' chunk/handoff/temp dirs root under **output.output_dir** (default **"output"**, byte-identical; default-OFF ? no Launch Report). Fixes the latent bug where scratch ignored **--output-dir**. Suite 1114 green, cov ~84.9%.

161 - 2026-06-21 ? **M4 landed** ([#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283)): **output.output_dir** (default **"output"**, byte-identical; default-OFF ? no Launch Report). Fixes the latent bug where scratch ignored **--output-dir**. Suite 1114 green, cov ~84.9%.

```

indexer/pull/283)): `DeviceContext` + native WASB **CPU-fallback** via a new `device="auto"`
(cuda-if-available-else-cpu); explicit `"cuda"` stays strict (no silent slow-CPU downgrade),
default unchanged. `"auto"` resolves before argv (never reaches `wasb_infer.py`); CUDA probe
cached. Device-selection tested offline; real CPU-inference run owner/GPU-verified. Suite 1126
green, cov 84.96%. **? M1?M4 (the config/input/output/device layer) COMPLETE.**
162

```

5. assistant (2026-06-21T09:25:07.060Z)

Now let me read the core files in my area. Let me batch these reads.

6. user (2026-06-21T09:25:13.625Z)

```

1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via `backend.storage` (a bare path /
8      `local://`
9      is an identity passthrough; `s3://` / `gcs://` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with `RALLY_DETECTOR_IMPL=stub` and
12     `--set indexing.skip_ai_handoff=true` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (`docs/SETUP_NEW_MACHINE.md`).
15
16     This is a thin orchestrator: it composes the SAME building blocks the `backend.main`
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import csv
24     import json
25     import logging
26     import os
27     import sys
28     import tempfile
29     from typing import Any, List
30
31     from backend import storage
32     from backend.config import load_config
33     from backend.infrastructure.database import Database
34     from backend.pipeline.segmenters.base import segmenter_registry
35     from backend.pipeline.validators.base import registry as validator_registry
36     from backend.reporting import emit_indexer_report
37     from backend.results import Failure
38     from backend.utils.hashing import compute_video_id
39
40     logger = logging.getLogger("backend.worker")
41
42     def _coerce(v: str) -> Any:
43         """Coerce a `--set` string value to bool/int/float where it round-trips, else
44         str."""
45         low = v.lower()
46         if low in ("true", "false"):
47             return low == "true"
48         for cast in (int, float):
49             try:
50                 return cast(v)
51             except ValueError:

```

```

48         pass
49     return v
50
51
52     def _apply_overrides(config: Any, sets: List[str]) -> None:
53         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
54         for kv in sets or []:
55             key, sep, val = kv.partition("=")
56             if not sep:
57                 raise ValueError(f"--set expects k=v, got {kv!r}")
58             parts = key.split(".")
59             obj = config
60             for p in parts[:-1]:
61                 obj = getattr(obj, p)
62             setattr(obj, parts[-1], _coerce(val))
63
64
65     def _write_intervals_csv(segments: List[dict], path: str) -> None:
66         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
67         artifact (same schema as the rally-annotator labels)."""
68         with open(path, "w", newline="") as f:
69             w = csv.writer(f)
70             w.writerow(["start", "end", "shot_count", "ending_reason"])
71             for s in segments:
72                 w.writerow([
73                     f'{float(s.get("start_time", 0.0)):.3f}',
74                     f'{float(s.get("end_time", 0.0)):.3f}',
75                     s.get("shot_count", ""),
76                     s.get("ending_reason", s.get("shot_count_confidence", "")),
77                 ])
78
79
80     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
81         with open(path, "w") as f:
82             json.dump({
83                 "video_id": video_id,
84                 "status": status,
85                 "segment_count": len(segments),
86                 "segments": [{"start": float(s.get("start_time", 0.0)),
87                             "end": float(s.get("end_time", 0.0))} for s in segments],
88             }, f, indent=2)
89
90
91     def cmd_infer(args: argparse.Namespace) -> int:
92         config = load_config(args.config) if args.config else load_config()
93         _apply_overrides(config, args.set)
94         storage_cfg = config.storage.model_dump()
95
96         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
97         os.makedirs(scratch, exist_ok=True)
98         config.output.output_dir = scratch
99
100        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
101        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
102        print(f"[worker] pulled {args.video_uri} -> {local_video}")
103
104        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
105        video_id = compute_video_id(local_video)
106

```

```

107     # 3. VALIDATE (same registry as the main CLI).
108     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
109     failures = [r for r in val_results if not r.passed]
110     db = Database(os.path.join(scratch, config.output.db_name))
111     db.add_video(video_id, local_video, "failed" if failures else "processed",
112                 [r.model_dump() for r in val_results])
113
114     segments: List[dict] = []
115     segmenter_actual = None
116     segmentation_ran = False
117     status = "failed"
118     try:
119         if failures:
120             print("[worker] validation FAILED: "
121                   + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
122             return 2
123         seg_name = args.segmenter or config.indexing.default_segmenter
124         segmenter_cls = segmenter_registry.get(seg_name)
125         if not segmenter_cls:
126             print(f"[worker] unknown segmenter: {seg_name!r} "
127                   f"(have {list(segmenter_registry.list_available())})")
128             return 2
129         segmenter_actual = seg_name
130         result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
131         segmentation_ran = True
132         if isinstance(result, Failure):
133             print(f"[worker] segmenter FAILED: {result.message}")
134             return 3
135         segments = result.value if hasattr(result, "value") else result
136         status = "success"
137         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
138         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
139         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
140         if not segments:
141             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
or getattr(config.indexing, "detector_impl", None) or
"auto")
142             logger.warning(
143                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
144                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
145                 "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
146                 "the WASB env produced no detections, or the input resolution differs
from the tuned "
147                 "proxy resolution. Re-run with -v for the native command + frame
counts.",
148                 video_id, segmenter_actual, detector_impl,
149             )
150         finally:
151             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
152             emit_indexer_report(
153                 db=db, video_id=video_id, video_path=local_video, config=config,
154                 val_results=val_results, val_context=val_context,
155                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
156                 was_reingest=False, segmentation_ran=segmentation_ran,
157             )
158     except:
159         pass

```

```

160     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
161     out_local = os.path.join(scratch, "outputs", video_id)
162     os.makedirs(out_local, exist_ok=True)
163     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
164     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
165
166     out_prefix = args.out_uri.rstrip("/")
167     pushed = []
168     for fn in sorted(os.listdir(out_local)):
169         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
170         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
171         pushed.append(uri)
172
173     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
174     for u in pushed:
175         print("    ", u)
176     return 0
177
178
179     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
180     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
181     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
182
183
184     def _probe_video_fps_width(path: str):
185         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
186
187         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
188         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
189         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
190         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
191         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
192         rather than guesses.
193         """
194         import json
195         import subprocess
196         try:
197             out = subprocess.check_output(
198                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
199                 "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
200                 stderr=subprocess.DEVNULL).decode()
201             st = (json.loads(out).get("streams") or [{}])[0]
202             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
203             fps = float(num) / float(den or "1")
204             width = float(st.get("width") or 0)
205             if fps > 0 and width > 0:
206                 return fps, width
207             except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
208                 pass
209             return None
210
211
212     def _resolve_train_detector(args: argparse.Namespace, config: Any):
213         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
214         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU

```

```

box,
215 stub=CPU mock). Returns the runner, or prints an error + returns None.
216
217 Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
218 detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
219 no shuttle detector and is rejected.
220 """
221 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
222
223 seg_cls = segmenter_registry.get(args.segmenter)
224 if not seg_cls:
225     print(f"[worker] unknown segmenter: {args.segmenter!r} "
226           f"(have {list(segmenter_registry.list_available())})")
227     return None
228 seg = seg_cls(None, config)
229 if not isinstance(seg, TrajectoryHybridSegmenter):
230     print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
231           f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
232     return None
233 try:
234     runner, _ = seg._resolve_runner(config.get("indexing", {}))
235 except NotImplementedError as e:
236     # e.g. detector_impl='native' for a family without a native runner (tracknet, or
237     # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
238     print(f"[worker] detector not available: {e}")
239     return None
240 ok, msg = runner.healthcheck()
241 if not ok:
242     print(f"[worker] detector environment not ready: {msg}")
243     return None
244 print(f"[worker] detector ready ({args.segmenter}): {msg}")
245 return runner
246
247
248 def cmd_train(args: argparse.Namespace) -> int:
249     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
250 to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
251
252 Two trajectory sources (the train pipeline + harness are identical either way):
253 * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
254 shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
255 * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
256 DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
257 is the M3.5 "first real training" path; only the trajectory source flips.
258 """
259 import subprocess
260
261 config = load_config(args.config) if args.config else load_config()
262 _apply_overrides(config, args.set)
263 storage_cfg = config.storage.model_dump()
264
265 scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
266 labels_dir = os.path.join(scratch, "labels")
267 traj_dir = os.path.join(scratch, "trajectories")
268 videos_dir = os.path.join(scratch, "videos")
269 os.makedirs(labels_dir, exist_ok=True)

```

```

270         os.makedirs(traj_dir, exist_ok=True)
271
272         corpus = args.corpus_uri.rstrip("/")
273         label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
274                             if u.lower().endswith(".csv"))
275         if len(label_uris) < 2:
276             print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
277                   f"under {corpus}/labels/")
278             return 2
279
280         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
281         runner = None
282         video_by_stem: dict = {}
283         if args.trajectory_source == "detector":
284             os.makedirs(videos_dir, exist_ok=True)
285             runner = _resolve_train_detector(args, config)
286             if runner is None:
287                 return 2
288             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
289                 vname = storage.parse_uri(vuri).name
290                 if not vname.lower().endswith(_VIDEO_EXTS):
291                     continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
292                 video_by_stem[os.path.splitext(vname)[0]] = vuri
293
294         print(f"[worker] trajectory source: {args.trajectory_source}")
295         specs: List[dict] = []
296         for uri in label_uris:
297             fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
298             name = fname.replace(".rallies.csv", "").replace(".csv", "")
299             local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
300             if args.trajectory_source == "detector":
301                 vuri = video_by_stem.get(name)
302                 if not vuri:
303                     print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
304                     continue
305                 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
306                                           config=storage_cfg)
307                 video_id = compute_video_id(local_video)
308                 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
309                 # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
310                 meta = _probe_video_fps_width(local_video)
311                 if meta is None:
312                     print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
313                     continue
314                 fps, frame_width = meta
315                 traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
316                 if not traj:
317                     print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
318                     continue
319                 specs.append({"name": name, "trajectory": traj, "labels": local_label,
320                             "fps": fps, "frame_width": frame_width})
321         else:
322             from backend.pipeline.detectors.stub_runner import

```

```

synth_trajectory_from_labels
323         traj = os.path.join(traj_dir, f"{name}_traj.csv")
324         synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
325         specs.append({"name": name, "trajectory": traj, "labels": local_label,
326                     "fps": args.fps, "frame_width": args.frame_width})
327
328     if len(specs) < 2:
329         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
330         return 2
331     manifest_path = os.path.join(scratch, "manifest.json")
332     with open(manifest_path, "w", encoding="utf-8") as f:
333         json.dump(specs, f, indent=2)
334     src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
335     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
336
337     out_dir = os.path.join(scratch, "artifacts")
338     cmd = [sys.executable, "-m", "training.gen0.harness",
339           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
340     if args.floor:
341         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
342                       if "://" in args.floor else args.floor)
343         cmd += ["--floor", floor_local]
344     print("[worker] training:", " ".join(cmd))
345     rc = subprocess.run(cmd).returncode
346     if rc != 0:
347         print(f"[worker] gen0 harness failed (rc={rc})")
348         return rc
349
350     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
351     bundle = os.path.join(out_dir, args.version)
352     out_prefix = args.out_uri.rstrip("/")
353     pushed = []
354     for fn in sorted(os.listdir(bundle)):
355         uri = f"{out_prefix}/weights/{args.version}/{fn}"
356         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
357         pushed.append(uri)
358     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
359     for u in pushed:
360         print("    ", u)
361     return 0
362
363
364 def build_parser() -> argparse.ArgumentParser:
365     p = argparse.ArgumentParser(prog="python -m backend.worker",
366                                description="Storage-aware worker: pull ? run pipeline ?
push.")
367     p.add_argument("-v", "--verbose", action="store_true",
368                   help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
369     sub = p.add_subparsers(dest="cmd", required=True)
370
371     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
372     pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
373     pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
374     pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
375     pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
376     pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp

```

```

dir)")
377     pi.add_argument("--max-frames", type=int, default=None)
378     pi.add_argument("--set", action="append", default=[], metavar="k=v",
379                     help="config override, e.g. --set indexing.skip_ai_handoff=true")
380     pi.set_defaults(func=cmd_infer)
381
382     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
383     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
384                     "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
385     pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>
is appended)")
386     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
387     pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
388                     help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
389                     "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
390                     "(needs videos/ in the corpus; honours detector_impl).")
391     pt.add_argument("--segmenter", default="wasb_hybrid",
392                     help="trajectory hybrid whose detector is used for --trajectory-
source detector")
393     pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
394     pt.add_argument("--config", default=None)
395     pt.add_argument("--scratch", default=None)
396     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
397     pt.add_argument("--frame-width", type=float, default=1280.0,
398                     help="synth trajectory frame width (detector probes the video)")
399     pt.add_argument("--set", action="append", default=[], metavar="k=v")
400     pt.set_defaults(func=cmd_train)
401     return p
402
403
404     def _setup_logging(verbose: bool) -> None:
405         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
406         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
407         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
408         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
409         entry), not at import, so importing the module never reconfigures a caller's
logging."""
410         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
411         # configured logging at import), which would leave the level at WARNING and re-
suppress the
412         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
413         logging.basicConfig(
414             level=logging.DEBUG if verbose else logging.INFO,
415             format="%(asctime)s [% (levelname)s] %(name)s: %(message)s",
416             datefmt="%H:%M:%S",
417             force=True,
418         )
419
420
421     def main(argv=None) -> int:
422         args = build_parser().parse_args(argv)
423         _setup_logging(getattr(args, "verbose", False))

```



```

48         return parse_uri(uri)
49
50
51     def input_is_local(raw: str, storage_config: Any = None) -> bool:
52         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55         ``resolve_input_ref``)."""
56         return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59     def acquire_local_input(raw: str, storage_config: Any = None, *,
60                             scratch_parent: "str | None" = None) -> str:
61         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66         path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67         regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68         worker's ``cmd_infer`` already ships."""
69         sc = _storage_dict(storage_config)
70         ref = resolve_input_ref(raw, sc)
71         if ref.backend == "local":
72             return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
73         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage
91         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94     def get_storage(backend: Optional[str] = None, *,
95                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97         the implementation; its per-backend settings come from ``config[backend]`` merged
with
98         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99         cfg = config or {}

```

```

100     name = backend or cfg.get("backend", "local")
101     settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102     settings.update(overrides)
103     return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107             config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
109         passthrough
110         (no copy); for s3/gcs it downloads to ``dst``."""
111         ref = parse_uri(uri)
112         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
113         overwrite=overwrite)
114
115
116     def put(local_path: "str | os.PathLike[str]", uri: str, *,
117             config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
118     str:
119         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
120         ref = parse_uri(uri)
121         get_storage(ref.backend, config=config).put(local_path, ref,
122         content_type=content_type)
123         return uri
124
125
126     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
127     Iterator[str]:
128         """Yield the URI of every object under ``prefix_uri``."""
129         ref = parse_uri(prefix_uri)
130         for r in get_storage(ref.backend, config=config).list(ref):
131             yield r.uri

```

8. assistant (2026-06-21T09:25:16.704Z)

Now let me read the job worker, uploads, and the database (jobs table / claim_due_job / WAIT_AND_RESUME).

9. user (2026-06-21T09:25:17.731Z)

```

1     """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
2     P3).
3
4     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
5     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
6     that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
7     re-claimed when due ? with no central scheduler (the DB is the clock).
8
9     The monkeypatch contract these functions rely on (and that tests depend on):
10    every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
11    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
12    `backend.main` module object at call-time (`import backend.main as _main`), never as a
13    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
14    ...) ` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
15    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
16    lazily
17    inside the functions also avoids the circular import at module load (main.py imports
18    this
19    module; this module must not import main at import time).
20    """
21    import logging
22    import threading
23    import traceback
24    from concurrent.futures import ThreadPoolExecutor

```

```

22     from typing import Any, Dict, Optional
23
24     logger = logging.getLogger(__name__)
25
26     # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
27     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
28     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
29     # already parallelize their AI handoff internally via their own pools.
30     _job_executor: Optional[ThreadPoolExecutor] = None
31
32
33     def _get_executor() -> ThreadPoolExecutor:
34         """Lazy module-global executor; tests monkeypatch this for inline execution."""
35         global _job_executor
36         if _job_executor is None:
37             _job_executor = ThreadPoolExecutor(max_workers=1,
38 thread_name_prefix="jobworker")
39         return _job_executor
40
41     class JobWaiting(Exception):
42         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
43         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
44         NOT a failure. The worker persists `next_poll_at` + `progress` (via
45 `set_job_waiting`),
46         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
47         date
48         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
49         contract.
50
51         The wiring is additive: the production segmenter path never raises this today (zero
52         behaviour change), so a job runs exactly as before. The hook is what a later slice
53         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
54         """
55
56         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
57             super().__init__(f"job parked for {delay_sec:.0f}s")
58             self.delay_sec = max(0.0, float(delay_sec))
59             self.progress = progress
60
61     def _job_to_request(job: Dict[str, Any]):
62         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
63         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
64         the original submit. The row stores exactly the ProcessRequest fields."""
65         import backend.main as _main
66         return _main.ProcessRequest(
67             video_path=job["video_path"],
68             player_pool=job.get("player_pool"),
69             max_frames=job.get("max_frames"),
70             segmenter=job.get("segmenter"),
71         )
72
73     def _run_claimed_job(job: Dict[str, Any]) -> None:
74         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
75         its
76         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
77
78         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
79         means
80         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
81         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
82         the
83         job self-scheduled and will be re-claimed when `next_poll_at` is due.

```

```

80     """
81     import backend.main as _main
82     db_instance = _main.db_instance
83     job_id = job["id"]
84     req = _main.job_to_request(job)
85     try:
86         result = _main._execute_processing(
87             req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88         if result.get("video_id"):
89             db_instance.set_job_video_id(job_id, result["video_id"])
90         db_instance.mark_job_done(job_id, result)
91     except _main.JobWaiting as w:
92         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
93         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
94         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
95     except Exception as e:
96         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
97         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
98
99
100     def _drain_due_jobs() -> int:
101         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
102         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
103
104         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
105         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
106         a single-worker box keeps making progress with no central timer (the DB is the
clock).
107         Returns the number of jobs that reached a terminal/parked state this tick.
108         """
109         import backend.main as _main
110         ran = 0
111         while True:
112             job = _main.db_instance.claim_due_job()
113             if job is None:
114                 break
115             ran += 1
116             _main._run_claimed_job(job)
117         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
118         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
119         # on the next submit/drain. Best-effort ? never raises into the worker.
120         _main._schedule_next_drain_if_waiting()
121         return ran
122
123
124     def _schedule_next_drain_if_waiting() -> None:
125         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
126         self-scheduled job resumes with no further client submit. The drain itself re-checks
127         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
128         import backend.main as _main
129         try:
130             delay = _main.db_instance.seconds_until_next_due()
131         except Exception as e: # pragma: no cover - defensive; never wedge the worker
132             logger.error(f"Could not compute next-due delay: {e}")
133             return
134         if delay is None:
135             return # nothing waiting
136         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-

```

```

ness.
137     _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
138
139
140     def _arm_wake_timer(delay_sec: float) -> None:
141         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
142         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
143         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
144         import backend.main as _main
145         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
146         timer.daemon = True
147         timer.start()
148
149
150     #: A parked job further out than this is re-checked by a capped wake rather than one
long
151     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
152     _MAX_DRAIN_WAKE_SEC = 60.0
153

```

10. user (2026-06-21T09:25:17.882Z)

```

1     """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3     Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4     proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5     under security.upload_dir, then submitted to /api/process by the returned path.
6     Upload state is in-process by design (this is the single-box alpha): a server restart
7     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8     arrives with PR-3 identity scoping.
9
10    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12    reads the live value the server set at startup (and so tests that monkeypatch
13    ``backend.main.global_config`` flow through unchanged).
14    """
15    import os
16    import threading
17    import uuid
18    from typing import Any, Dict, Optional
19
20    from fastapi import APIRouter, HTTPException, Request
21    from pydantic import BaseModel
22
23    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
24    from backend.utils.paths import safe_output_filename
25
26    router = APIRouter()
27
28    _uploads: Dict[str, Dict[str, Any]] = {}
29    _uploads_lock = threading.Lock()
30
31
32    class UploadInitRequest(BaseModel):
33        filename: str
34        total_size_bytes: Optional[int] = None
35
36
37    class UploadCompleteRequest(BaseModel):
38        total_parts: int
39
40

```

```

41     def _upload_cfg():
42         # Resolved lazily (import inside the function) to read the LIVE startup config that
43         # main() sets on the module global, and to avoid a circular import at module load
44         # (main.py imports this router; this module must not import main at import time).
45         import backend.main as _main
46         sec = getattr(_main.global_config, "security", None)
47         upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
48         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
49         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
50         exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
51         None) or ["mp4", "mov"])]
52
53     def _abort_upload(upload_id: str) -> None:
54         with _uploads_lock:
55             st = _uploads.pop(upload_id, None)
56             if st:
57                 try:
58                     os.remove(st["tmp_path"])
59                 except OSError:
60                     pass
61
62     @router.post("/api/upload/init")
63     def upload_init(req: UploadInitRequest):
64         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
65         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
66         try:
67             name = safe_output_filename(req.filename)
68         except ValueError as e:
69             raise HTTPException(status_code=400, detail=str(e)) from e
70         ext = os.path.splitext(name)[1].lower().lstrip(".")
71         if ext not in exts:
72             raise HTTPException(status_code=400,
73             detail=f"Extension '{ext}' not allowed. Allowed:
74             {sorted(exts)}")
75         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
76             raise HTTPException(status_code=413,
77             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
78             upload limit.")
79         os.makedirs(upload_dir, exist_ok=True)
80         upload_id = uuid.uuid4().hex
81         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
82         open(tmp_path, "wb").close()
83         with _uploads_lock:
84             _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
85             "bytes": 0}
86         return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
87             "next_part": f"/api/upload/{upload_id}/part/0"}
88
89     @router.put("/api/upload/{upload_id}/part/{index}")
90     async def upload_part(upload_id: str, index: int, request: Request):
91         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
92         _, max_bytes, part_bytes, _ = _upload_cfg()
93         with _uploads_lock:
94             st = _uploads.get(upload_id)
95             if st is None:
96                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
97             if index != st["parts"]:
98                 raise HTTPException(status_code=409,
99                 detail=f"Out-of-order part: expected {st['parts']}, got
100                 {index} (parts are sequential)")

```

```

101         received = 0
102         overflow = None
103         with open(st["tmp_path"], "ab") as fh:
104             async for chunk in request.stream():
105                 received += len(chunk)
106                 if received > part_bytes:
107                     overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
108                     break
109                 if st["bytes"] + received > max_bytes:
110                     overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
111                     break
112                 fh.write(chunk)
113         if overflow:
114             _abort_upload(upload_id)
115             raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
116
117         with _uploads_lock:
118             st["parts"] += 1
119             st["bytes"] += received
120         return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
121
122
123     @router.post("/api/upload/{upload_id}/complete")
124     def upload_complete(upload_id: str, req: UploadCompleteRequest):
125         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
126         upload_dir, _, _, _ = _upload_cfg()
127         with _uploads_lock:
128             st = _uploads.get(upload_id)
129         if st is None:
130             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
131         if st["parts"] == 0 or st["parts"] != req.total_parts:
132             raise HTTPException(status_code=409,
133                                 detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
134         final_path = os.path.join(upload_dir, st["filename"])
135         if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
136             stem, ext = os.path.splitext(st["filename"])
137             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138         os.replace(st["tmp_path"], final_path)
139         with _uploads_lock:
140             _uploads.pop(upload_id, None)
141         video_id = compute_video_id(final_path)
142         return {"path": os.path.abspath(final_path), "video_id": video_id,
143                 "size_bytes": os.path.getsize(final_path),
144                 "next": "POST /api/process with this path"}
145
146
147     @router.delete("/api/upload/{upload_id}")
148     def upload_abort(upload_id: str):
149         """Abort an in-flight upload and delete its partial file."""
150         with _uploads_lock:
151             known = upload_id in _uploads
152         if not known:
153             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
154         _abort_upload(upload_id)
155         return {"status": "aborted", "upload_id": upload_id}
156

```

Now let me read the database (jobs table, claim_due_job CAS, ExecutionPolicy/JobWaiting, migration ladder) and the `\_resolve\_runner` compute seam.

12. user (2026-06-21T09:25:22.849Z)

```
67:         ``user_version`` stays un-bumped ? and the next open would re-run the same
86:         # Versioned migrations live here. PRAGMA user_version is the schema
89:         version = cursor.execute("PRAGMA user_version").fetchone()[0]
105:         # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA user_version
= 6
178:         cursor.execute("PRAGMA user_version = 1")
205:         cursor.execute("PRAGMA user_version = 2")
209:         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is due
211:         # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
215:         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
216:         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
217:         cursor.execute("PRAGMA user_version = 3")
237:         cursor.execute("PRAGMA user_version = 4")
275:         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
285:         cursor.execute("PRAGMA user_version = 5")
490:         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
552:         """Attach/replace a job's JSON ExecutionPolicy."""
559:         def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] = None)
-> bool:
561:             worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
565:             "next_poll_at=datetime('now', ?), "
571:             def seconds_until_next_due(self) -> Optional[float]:
574:                 already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
578:                 "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
579:                 " (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS secs "
588:             def claim_due_job(self) -> Optional[Dict[str, Any]]:
589:                 """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose ``next_poll_at``
is due ?
598:                 "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
599:                 "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1").fetchone()
```

13. user (2026-06-21T09:25:23.376Z)

```
1         """Shared engine for trajectory-substrate hybrid segmenters.
2
3         tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4         standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5         now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10        A concrete subclass supplies only its identity and runner wiring:
11        - ``family`` ? config section under ``indexing.*``, output subdir, and
12          the ``<family>-hybrid-<model>`` detector tag in metadata.
13        - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14        - ``name`` / ``description`` properties (the registry contract).
15        - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16          detection_params extras such as weights_path).
17
18        This base is deliberately NOT registered; only concrete subclasses register.
19        """
20        import os
21        import time
```

```

22     import logging
23     import concurrent.futures
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.utils.paths import output_base
31     from backend.sports import sport_registry
32     from backend.providers import provider_registry
33     from backend.pipeline.detectors.base import DetectorRunner
34
35     logger = logging.getLogger(__name__)
36
37
38     def _fmt_ts(seconds: float) -> str:
39         seconds = max(0, int(seconds))
40         h, rem = divmod(seconds, 3600)
41         m, s = divmod(rem, 60)
42         return f"{h:02d}:{m:02d}:{s:02d}"
43
44
45     class TrajectoryHybridSegmenter(BasePlaySegmenter):
46         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
47         boundaries."""
48
49         #: config section under `indexing` (velocity_threshold lives there), the
50         #: output subdir for trajectories, and the metadata detector-tag prefix.
51         family: str = ""
52         #: human-readable label used in log lines.
53         display_label: str = ""
54
55         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
56         Any]]:
57             """Return (runner, detector-specific detection_params extras) for the default
58             (WSL) path."""
59             raise NotImplementedError
60
61         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
62         Dict[str, Any]]:
63             """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
64             with a
65             native runner override this; the base refuses with a clear message (M3.5: WASB
66             only)."""
67             raise NotImplementedError(
68                 f"detector_impl='native' is not supported for the "
69                 f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
70                 has a "
71                 f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
72
73         def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
74         Dict[str, Any]]:
75             """Resolve the detector runner, honouring the CPU-stub and Linux-native
76             overrides.
77
78             ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
79             ``indexing.detector_impl``:
80             ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
81             pipeline
82             is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
83             swaps
84             in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
85             ``_build_native_runner``. Anything else (``"auto"``) delegates to
86             ``_build_runner``

```

```

74         (the WSL runner) so the default production path is unchanged
75         (`docs/DECOUPLED_COMPUTE/`).
76         """
77         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
78         if impl == "stub":
79             from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
80             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
81                         self.display_label or "trajectory")
82             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
83         if impl in ("native", "native_wasb", "wasb_native"):
84             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
85                         self.display_label or "trajectory")
86             return self._build_native_runner(idx_cfg)
87         return self._build_runner(idx_cfg)
88
89     @staticmethod
90     def _probe_fps_width(video_path: str) -> tuple:
91         """Return (fps, frame_width) for a video, with sensible fallbacks."""
92         cap = cv2.VideoCapture(video_path)
93         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
94         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
95         cap.release()
96         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
97
98     @staticmethod
99     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
100         """Provider-reported exchange count, else a duration-based estimate.
101
102         One canonical implementation ? the twins had drifted (wasb relied on
103         ``int(None)`` raising into the fallback; same result, by accident).
104         """
105         shot_count = segment.get("shuttle_exchange_count")
106         if shot_count is None:
107             return max(3, int(duration / 0.8))
108         try:
109             return int(shot_count)
110         except (ValueError, TypeError):
111             return max(3, int(duration / 0.8))
112
113     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
114     -----
115     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
116                                frame_width: float, video_path: str,
117                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
118         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
118         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
119         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
120         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""

```

14. assistant (2026-06-21T09:25:26.476Z)

Now let me read the jobs table schema and the key job methods in detail.

15. user (2026-06-21T09:25:27.847Z)

```

180     def _migrate_to_v2(self, cursor):
181         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
182         # /api/process request. `status` is the EXECUTION state of the job
183         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video

```

```

184         # that failed validation) lives inside `result` JSON as result["status"].
185         # `result` is the full sync-era response dict (incl. report paths), so the
186         # observability report is the job's artifact, per the plan.
187         cursor.execute("""
188             CREATE TABLE IF NOT EXISTS jobs (
189                 id TEXT PRIMARY KEY,
190                 video_path TEXT NOT NULL,
191                 video_id TEXT,
192                 segmenter TEXT,
193                 player_pool TEXT,
194                 max_frames INTEGER,
195                 status TEXT NOT NULL DEFAULT 'queued',
196                 progress TEXT,
197                 error TEXT,
198                 result TEXT,
199                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
200                 started_at TIMESTAMP,
201                 finished_at TIMESTAMP
202             )
203         """)
204         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
205         cursor.execute("PRAGMA user_version = 2")
206
207     def _migrate_to_v3(self, cursor):
208         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
209         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
210         # due
211         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
212         # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
213         # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
214         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
215         # them.
216         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
217         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
218         TIMESTAMP")
219         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
220         next_poll_at)")
221         cursor.execute("PRAGMA user_version = 3")
222
223     def _migrate_to_v4(self, cursor):
224         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
225         # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
226         # path can scope rows to an owner. **NULL = local install = byte-identical to
227         # today** ? every existing row stays NULL and every existing query (which never
228         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
229         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
230         # them.
231         # Additive ONLY: no served-behavior change rides on this slice.
232         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
233         TEXT")
234         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
235         user_id TEXT")
236         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
237         # of index churn while making the eventual per-user lookups cheap.
238         cursor.execute(
239             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
240             "WHERE user_id IS NOT NULL")
241         cursor.execute(
242             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
243             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
244         cursor.execute("PRAGMA user_version = 4")
245
246     def _migrate_to_v5(self, cursor):
247         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
248         # version): the resolved per-user `fusion_config` overlay + an INLINE

```

```

242         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
243         # and provenance that earned it. `is_active` pins the one row the serving bridge
244         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
245         # only persists/loads it ? NOTHING reads it on the served path yet.
246         #
247         #     user_id            ? owner of the model (NULL allowed = the local install)
248         #     version            ? monotonic per-user model version
249         #     baseline_version    ? the shared baseline this was fit against (upgrade
switch)
250         #     feature_schema_hash ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
251         #     fusion_config      ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
252         #     classifier_json     ? JSON LogisticRegression.to_dict() (None = config-only
model)
253         #     promotion_evidence  ? JSON per_user_gate PromotionDecision (why it was
promoted)
254         #     n_videos_at_fit     ? held-out-user corpus size when fit (min_n trust floor
input)
255         #     is_active          ? the one resolved row for this user (partial-unique
below)
256         cursor.execute("""
257             CREATE TABLE IF NOT EXISTS user_models (
258                 id INTEGER PRIMARY KEY AUTOINCREMENT,
259                 user_id TEXT,
260                 version INTEGER NOT NULL DEFAULT 1,
261                 baseline_version TEXT,
262                 feature_schema_hash TEXT,
263                 fusion_config TEXT,
264                 classifier_json TEXT,
265                 promotion_evidence TEXT,
266                 n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
267                 is_active INTEGER NOT NULL DEFAULT 0,
268                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
269             )
270         """)
271         # One model version per user (NULLs compare distinct in SQLite, so the local
272         # install can still carry multiple versioned rows; the active-row guard below
273         # is what enforces a single served model).
274         cursor.execute("""
275             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
276             ON user_models(user_id, version)
277         """)
278         # At most ONE active model per user: a partial-unique index over the active
rows.
279         # (SQLite treats NULL user_id rows as distinct, so the local install's single

```

16. user (2026-06-21T09:25:28.623Z)

```

480         conn.commit()
481
482         # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
483         # Writers follow the repo convention (swallow + logger.error, return bool); the
484         # worker logs loudly on a failed transition so a job can't silently wedge.
485
486         def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
487                         player_pool: Optional[List[str]] = None,
488                         max_frames: Optional[int] = None,
489                         policy: Optional[Dict[str, Any]] = None) -> bool:
490             """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
491             (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
492             return self._execute_write(

```

```

493         "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy) "
494         "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
495         (job_id, video_path, segmenter,
496         json.dumps(player_pool) if player_pool is not None else None,
497         max_frames, json.dumps(policy) if policy is not None else None),
498         f"Failed to create job {job_id}",
499     )
500
501     def mark_job_running(self, job_id: str) -> bool:
502         return self._execute_write(
503             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
504             "progress='starting' WHERE id = ?", (job_id,),
505             f"Failed to mark job {job_id} running",
506         )
507
508     def set_job_progress(self, job_id: str, progress: str) -> bool:
509         return self._execute_write(
510             "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
511             f"Failed to set progress for job {job_id}",
512         )
513
514     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
515         return self._execute_write(
516             "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
517             f"Failed to set video_id for job {job_id}",
518         )
519
520     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
521         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
522         validation) is inside `result` ? job status 'done' means 'the worker
523         finished'."""
524         return self._execute_write(
525             "UPDATE jobs SET status='done', progress='finished', result=?, "
526             "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
527             (json.dumps(result), job_id),
528             f"Failed to mark job {job_id} done",
529         )
530
531     def mark_job_failed(self, job_id: str, error: str) -> bool:
532         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
533         return self._execute_write(
534             "UPDATE jobs SET status='failed', error=?, "
535             "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
536             f"Failed to mark job {job_id} failed",
537         )
538
539     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
540         """Fetch one job; `result`/`player_pool` JSON deserialized."""
541         with self._connect() as conn:
542             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
543             (job_id,)).fetchone()
544             if not row:
545                 return None
546             d = dict(row)
547             d["result"] = json.loads(d["result"]) if d["result"] else None
548             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
549             None
550             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
551             return d
552
553     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
554     -----
555     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
556         """Attach/replace a job's JSON ExecutionPolicy."""

```

```

553         return self._execute_write(
554             "UPDATE jobs SET policy=? WHERE id = ?",
555             (json.dumps(policy) if policy is not None else None, job_id),
556             f"Failed to set policy for job {job_id}",
557         )
558
559     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
560         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
561         worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
562         computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
563         return self._execute_write(
564             "UPDATE jobs SET status='waiting', "
565             "next_poll_at=datetime('now', ?), "
566             "progress=COALESCE(?, progress) WHERE id = ?",
567             (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
568             f"Failed to set job {job_id} waiting",
569         )
570
571     def seconds_until_next_due(self) -> Optional[float]:
572         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
573         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
574         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
575         try:
576             with self._connect() as conn:
577                 row = conn.cursor().execute(
578                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
579                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
580                     "FROM jobs WHERE status='waiting'").fetchone()
581                 if row is None or row["secs"] is None:
582                     return None
583                 return max(0.0, float(row["secs"]))
584         except Exception as e:
585             logger.error(f"Failed to compute next-due delay: {e}")
586             return None
587
588     def claim_due_job(self) -> Optional[Dict[str, Any]]:
589         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
590         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
591         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
592         first. Returns the claimed job dict, or None if nothing is runnable."""
593         try:
594             with self._connect() as conn:
595                 cur = conn.cursor()
596                 row = cur.execute(
597                     "SELECT id, status FROM jobs WHERE status='queued' "
598                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
599                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
600                 if not row:
601                     return None
602                 cur.execute(
603                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
604                     "WHERE id=? AND status=?", (row["id"], row["status"]))

```

```

605             conn.commit()
606             if cur.rowcount != 1:
607                 return None                    # lost the race to a concurrent
worker
608             return self.get_job(row["id"])
609         except Exception as e:

```

17. assistant (2026-06-21T09:25:33.442Z)

I now have a clear picture of the job control plane. Let me look at the GCS storage backend and the migration ladder dispatcher to confirm what exists for the cloud substrate, plus check the worker `\_build\_native\_runner` (the cloud detector path) and any existing dispatch/Cloud Run code.

18. user (2026-06-21T09:25:34.464Z)

```

1      """Google Cloud Storage backend ? first-class object buckets.
2
3      Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4      install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5      (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6      never from config. A ``client`` may be injected for tests.
7      """
8      from __future__ import annotations
9
10     from datetime import timedelta
11     from typing import Any, Iterator, Optional
12
13     from backend.storage.base import StorageBackend
14     from backend.storage.ref import StorageRef, StorageStat
15
16
17     class GCSStorage(StorageBackend):
18         def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                     signed_url_ttl: int = 3600, **_ignored) -> None:
20             self._ttl = signed_url_ttl
21             if client is not None:
22                 self._client = client
23             return
24             from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25             self._client = storage.Client(project=project)
26
27         def _blob(self, ref: StorageRef):
28             return self._client.bucket(ref.bucket).blob(ref.key)
29
30         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31             if dst is None:
32                 raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33             import os
34             dst = str(dst)
35             if os.path.exists(dst) and not overwrite:
36                 return dst
37             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38             self._blob(ref).download_to_filename(dst)
39             return dst
40
41         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42             self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43             return ref
44
45         def exists(self, ref) -> bool:

```

```

46         return bool(self._blob(ref).exists())
47
48     def stat(self, ref) -> StorageStat:
49         b = self._blob(ref)
50         b.reload()
51         return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                             content_type=b.content_type)
53
54     def list(self, prefix) -> Iterator[StorageRef]:
55         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56             yield StorageRef("gcs", prefix.bucket, blob.name,
57 f"gcs://{prefix.bucket}/{blob.name}")
58
59     def delete(self, ref) -> None:
60         self._blob(ref).delete()
61
62     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
63         return self._blob(ref).generate_signed_url(
64             version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)

```

19. user (2026-06-21T09:25:35.220Z)

```

85
86     # Versioned migrations live here. PRAGMA user_version is the schema
87     # contract ? bump it for every change and add an ordered `if version < N`
88     # block below. Tests assert the expected version, so accidental drift fails
89
90     CI.
91     version = cursor.execute("PRAGMA user_version").fetchone()[0]
92
93     if version < 1:
94         self._migrate_to_v1(cursor)
95
96     if version < 2:
97         self._migrate_to_v2(cursor)
98
99     if version < 3:
100         self._migrate_to_v3(cursor)
101
102     if version < 4:
103         self._migrate_to_v4(cursor)
104
105     if version < 5:
106         self._migrate_to_v5(cursor)
107     # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
108 user_version = 6
109
110     conn.commit()
111
112     def _create_base_tables(self, cursor):
113         """Create the videos / play_segments / events base tables (idempotent)."""
114         # Videos Table
115         cursor.execute("""
116             CREATE TABLE IF NOT EXISTS videos (
117                 id TEXT PRIMARY KEY,
118                 filepath TEXT NOT NULL,
119                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
120                 status TEXT NOT NULL,
121                 validation_results TEXT
122             )
123         """)
124
125         # Play Segments Table (Extensible metadata JSON)
126         cursor.execute("""
127             CREATE TABLE IF NOT EXISTS play_segments (

```

```

125         id INTEGER PRIMARY KEY AUTOINCREMENT,
126         video_id TEXT NOT NULL,
127         start_time REAL NOT NULL,
128         end_time REAL NOT NULL,
129         duration REAL NOT NULL,
130         server TEXT,
131         receiver TEXT,
132         metadata TEXT,
133         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
134     )
135 """
136
137 # Events Table
138 cursor.execute("""
139     CREATE TABLE IF NOT EXISTS events (
140         id INTEGER PRIMARY KEY AUTOINCREMENT,
141         segment_id INTEGER NOT NULL,
142         timestamp REAL NOT NULL,
143         type TEXT NOT NULL,
144         player TEXT,
145         details TEXT,
146         FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
147     )
148 """
149
150 def _migrate_to_v1(self, cursor):
151     # v1: raw candidate detections (pre-confirmation), detector-agnostic.
152     # Common fields are columns; detector-specific metrics go in `stats` JSON,
153     # so a new segmenter reuses this table by setting its own `detector`/`stats`.
154     cursor.execute("""
155         CREATE TABLE IF NOT EXISTS candidate_segments (
156             id INTEGER PRIMARY KEY AUTOINCREMENT,
157             video_id TEXT NOT NULL,
158             detector TEXT NOT NULL,
159             chunk_index INTEGER,
160             start_time REAL NOT NULL,
161             end_time REAL NOT NULL,
162             duration REAL NOT NULL,
163             confidence REAL,
164             stats TEXT,
165             detection_params TEXT,
166             confirmed_segment_id INTEGER,
167             human_label TEXT,
168             notes TEXT,
169             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
170             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
171             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
172         )
173     """
174     cursor.execute("""
175         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
176             ON candidate_segments(video_id, detector)
177     """
178     cursor.execute("PRAGMA user_version = 1")
179
180 def _migrate_to_v2(self, cursor):
181     # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
182     # /api/process request. `status` is the EXECUTION state of the job
183     # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
184     # that failed validation) lives inside `result` JSON as result["status"].
185     # `result` is the full sync-era response dict (incl. report paths), so the
186     # observability report is the job's artifact, per the plan.
187     cursor.execute("""
188         CREATE TABLE IF NOT EXISTS jobs (

```

```

189             id TEXT PRIMARY KEY,
190             video_path TEXT NOT NULL,
191             video_id TEXT,
192             segmenter TEXT,
193             player_pool TEXT,
194             max_frames INTEGER,
195             status TEXT NOT NULL DEFAULT 'queued',
196             progress TEXT,
197             error TEXT,
198             result TEXT,
199             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
200             started_at TIMESTAMP,
201             finished_at TIMESTAMP
202         )
203     """
204     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")

```

20. assistant (2026-06-21T09:25:38.317Z)

Now let me check what already exists for Cloud Run dispatch and Dockerfiles, plus the `\_execute\_processing` path and the native WASB runner, to confirm what's net-new.

21. user (2026-06-21T09:25:41.836Z)

```

deploy/digitalocean/bootstrap.sh
deploy/digitalocean/gpu_setup.sh
deploy/digitalocean/mount_scratch.sh
deploy/digitalocean/README.md
deploy/gcp/provision.sh
deploy/gcp/README.md
deploy/gcp/teardown.sh
---DOCKERFILES---
---CLOUD RUN / DISPATCH REFS---
no backend/compute dir

```

22. user (2026-06-21T09:25:42.996Z)

```

Found 8 files
backend\config\models.py
backend\storage\__init__.py
backend\config\loader.py
backend\config\presets.py
backend\worker\__main__.py
backend\storage\base.py
backend\worker\__init__.py
backend\api\job_worker.py

```

23. assistant (2026-06-21T09:25:47.133Z)

No Dockerfile exists, no `backend/compute` dir, and `deploy/gcp` only has VM provisioning scripts (not Cloud Run). Let me confirm the `compute\_target` config wiring and the deploy/gcp scripts to ground what's there, and check the native WASB runner + healthcheck (the container's detector).

24. user (2026-06-21T09:25:48.662Z)

```

18:class ValidationRelevanceConfig(BaseModel):
24:class ValidationConfig(BaseModel):
33:class TrackNetConfig(BaseModel):
36:     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
54:class WasbIndexConfig(BaseModel):
63:     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
94:class FusionConfig(BaseModel):
127:class IndexingConfig(BaseModel):

```

```

230:class OutputConfig(BaseModel):
238:class WatermarkConfig(BaseModel):
255:class StitchingConfig(BaseModel):
271:class LoggingConfig(BaseModel):
280:class SecurityConfig(BaseModel):
297:class StorageConfig(BaseModel):
334:class DeploymentConfig(BaseModel):
335:    """Deployment profile ? one switch that selects a coherent bundle of compute + storage
338:    **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
339:    behaves exactly as before this section existed. A profile is opt-in (``config.json``
340:    ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
342:    these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
347:    # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
348:    profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
349:    # Where the core runs. Set by the profile preset; overridable in config.json. Inert in
M1
351:    compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
354:class AppConfig(BaseModel):

```

25. user (2026-06-21T09:25:49.619Z)

```
#!/usr/bin/env bash
```

Provision the DECOUPLED_COMPUTE GCP storage + IAM (asia-south1 / Mumbai). Ide
re-run; already-existing resources are left as-is. The GPU VM is NOT created here (it is
the GPUS_ALL_REGIONS quota ? see deploy/gcp/README.md §2-3); create it with th
the quota is granted. This documents EXACTLY what was run on 2026-06-20 (DO->GC

Prereq: `gcloud auth login`; Service Usage API + a linked billing account enabled in th
(README §1). Run: bash deploy/gcp/provision.sh

```
set -uo pipefail
```

```

PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
REGION="${RALLY_GCP_REGION:-asia-south1}"
BUCKET="${RALLY_GCS_BUCKET:-khelsutra-rally-corpus}"
BILLING_ACCT="${RALLY_BILLING_ACCT:-01420D-419EE0-53D83C}"
SA_NAME="rally-worker"
SA_EMAIL="${SA_NAME}@${PROJECT}.iam.gserviceaccount.com"
SA_KEY="${RALLY_SA_KEY:-$HOME/.gcp/rally-worker-sa.json}"
export CLOUDSDK_CORE_DISABLE_PROMPTS=1

```

```
echo "== project=$PROJECT region=$REGION bucket=gs://$BUCKET =="
```

```
echo "== enable APIs =="
```

```

gcloud services enable compute.googleapis.com cloudbilling.googleapis.com \
    billingbudgets.googleapis.com iam.googleapis.com cloudresourcemanager.googleapis.com \
    --project "$PROJECT"

```

```
echo "== GCS bucket (asia-south1, uniform bucket-level access) =="
```

```

gcloud storage buckets create "gs://$BUCKET" --location="$REGION" \
    --uniform-bucket-level-access --project "$PROJECT" || echo " (bucket exists ? ok)"

```

```
echo "== service account + Storage Object Admin + key =="
```

```

gcloud iam service-accounts create "$SA_NAME" --display-name="Rally worker (storage)" \
    --project "$PROJECT" || echo " (SA exists ? ok)"
gcloud projects add-iam-policy-binding "$PROJECT" \
    --member="serviceAccount:$SA_EMAIL" --role="roles/storage.objectAdmin" --condition=None \
    --format="value(etag)" >/dev/null

```

```
if [ ! -f "$SA_KEY" ]; then
```

```
mkdir -p "$(dirname "$SA_KEY")"
gcloud iam service-accounts keys create "$SA_KEY" --iam-account="$SA_EMAIL"
echo " key -> $SA_KEY (NEVER commit this; export GOOGLE_APPLICATION_CREDENTIALS=$SA_KEY)"
===README===
```

GCP provisioning runbook ? DECOUPLED_COMPUTE GPU + storage (asia-south1)

```
> **? GCP is the SOLE compute stack (ADR-013, 2026-06-20).** DigitalOcean is **dropped for
compute**
> (DO GPU Droplets aren't enabled on the account + aren't cleanly credit-funded ? DO charges
newer
> customers' cards for GPU; Paperspace is subscription-gated). **All training + serving runs on
GCP.**
> The `deploy/digitalocean/` scripts are kept only as **provider-agnostic** Linux-GPU setup that
this
> runbook reuses. ? Capacity note: asia-south1 (Mumbai) GPUs are frequently stocked-out ? sweep
zones /
> fall back to asia-southeast1 (Singapore) etc.; weights still land in the asia-south1 bucket.
```

Co-located **GPU VM + GCS bucket in ONE region** (asia-south1 / Mumbai) for the first real (non-mock) WASB training + inference. This is the **DO?GCP pivot** (ADR-010): DigitalOcean GPU Droplets are North-America-only (no Singapore/India GPU), so we moved GPU + storage to GCP, which has GPUs in asia-south1 (Mumbai) and asia-southeast1 (Singapore). Owner picked **Mumbai** (closest to Bangalore ? lowest latency).

```
> **Cost discipline (read first):** a running GPU VM is the only meaningful cost. Use the
cheapest
> adequate GPU (**T4**), **STOP or DELETE the VM the moment a run finishes** (a stopped VM bills
only
> for its disk), and keep the **$100 budget alert** live. `teardown.sh` switches everything off.
```

0. Live resources (created 2026-06-20 ? the "switch-off" inventory)

Resource	Identifier	Cost when idle	How to switch off
Project	`gen-lang-client-0306026826` (billing Khelsutra Billing `01420D-419EE0-53D83C`)		
? n/a (shared Gemini project)			
GCS bucket	`gs://khelsutra-rally-corpus` (asia-south1, UBLA) ~\$0.02/GB/mo storage		
gcloud storage rm -r` (? data loss)			
Service account	`rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com`		
(`roles/storage.objectAdmin`) \$0 `gcloud iam service-accounts delete`			
SA key	`~/gcp/rally-worker-sa.json` (local only, never committed) \$0 delete the file + the key		
Budget alert	`rally-100usd-guard` ? \$100, alerts at 50/90/100% \$0 `gcloud billing budgets delete`		

26. assistant (2026-06-21T09:25:53.258Z)

I have enough to map M6 and M7. Let me confirm the native WASB runner healthcheck (the container's detector) and check `\_execute\_processing` and the GCS signed_url (for M7's reel delivery) exist, plus the presets for cloud-serving.

27. user (2026-06-21T09:25:54.848Z)

```
4:config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28:COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
34:    "cloud-serving": "cloud_run",
35:    "cpu-mock": "cpu_mock",
40:# ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
```

```

extend
48:     "indexing": {"skip_ai_handoff": True},
51:     "cloud-serving": {
52:         "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
54:         "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
58:         "deployment": {"profile": "cpu_mock", "compute_target": "cpu_mock"},
60:         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
104:     - ``K_SERVICE`` set (Cloud Run)      ? ``cloud-serving``
110:     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
115:         return "cloud-serving"

```

28. user (2026-06-21T09:25:56.421Z)

```

backend\pipeline\detectors\__init__.py:7:- wasb_infer.py (the actual code that runs *inside* the
WSL wasb env).
backend\pipeline\detectors\wasb_runner.py:3:Windows-side counterpart to
backend\detectors\wasb_infer.py. It:
backend\pipeline\detectors\wasb_runner.py:5: 2. copies wasb_infer.py into the WASB repo's src/
dir,
backend\pipeline\detectors\wasb_runner.py:10:with the TrackNet path. Verified inference core:
wasb_infer.py (see its docstring).
backend\pipeline\detectors\wasb_runner.py:27:_INFER_WIN =
os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
backend\pipeline\detectors\wasb_runner.py:72:     def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\wasb_runner.py:107:         """Copy wasb_infer.py into the WASB repo
src/ so it can import WASB modules."""
backend\pipeline\detectors\wasb_runner.py:109:         cmd = f"cp {shlex.quote(src_mnt)}
{self.cfg.repo_dir}/src/wasb_infer.py && echo OK"
backend\pipeline\detectors\wasb_runner.py:113:     def run_predict(self, video_win_path: str,
output_win_dir: str,
backend\pipeline\detectors\wasb_runner.py:137:         logger.error("Could not stage
wasb_infer.py into WSL.")
backend\pipeline\detectors\wasb_runner.py:163:         f"python wasb_infer.py "
backend\pipeline\detectors\native_wasb_runner.py:6:Linux GPU box (GCP / a SaaS worker): it runs
the **same** verified ``wasb_infer.py`` core
backend\pipeline\detectors\native_wasb_runner.py:15:     strict); threaded into ``wasb_infer.py
--device`` (and so the Hydra ``runner.device`` override).
backend\pipeline\detectors\native_wasb_runner.py:52:_INFER_PY =
os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
backend\pipeline\detectors\native_wasb_runner.py:100:class NativeWasbRunner(DetectorRunner):
backend\pipeline\detectors\native_wasb_runner.py:132:         """The torch device to actually
pass to ``wasb_infer.py``. Only ``auto`` needs the CUDA
backend\pipeline\detectors\native_wasb_runner.py:142:         """Copy wasb_infer.py into the WASB
repo src/ so it imports WASB modules + finds configs/."""
backend\pipeline\detectors\native_wasb_runner.py:146:         shutil.copy(_INFER_PY,
os.path.join(repo_src, "wasb_infer.py"))
backend\pipeline\detectors\native_wasb_runner.py:149:         logger.error("Failed to copy
wasb_infer.py into %s: %s", repo_src, e)
backend\pipeline\detectors\native_wasb_runner.py:157:     def healthcheck(self) -> Tuple[bool,
str]:
backend\pipeline\detectors\native_wasb_runner.py:169:         # argparse rc=2 from wasb_infer.py
at run time.
backend\pipeline\detectors\native_wasb_runner.py:193:     def run_predict(self, video_win_path:
str, output_win_dir: str,
backend\pipeline\detectors\native_wasb_runner.py:211:         logger.error("Could not stage
wasb_infer.py into the WASB repo src/.")
backend\pipeline\detectors\native_wasb_runner.py:215:         argv = [self.cfg.python_bin,
"wasb_infer.py",
backend\pipeline\detectors\native_wasb_runner.py:226:         # explicit cuda/mps/cpu passes
through unchanged. Never sends "auto" to wasb_infer.py.
backend\pipeline\detectors\stub_runner.py:70:     def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\stub_runner.py:97:     def run_predict(self, video_win_path: str,
output_win_dir: str,
backend\pipeline\detectors\wasb_infer.py:36:     python wasb_infer.py --frames_dir /path/frames

```

```
--weights ../pretrained_weights/wasb_badminton_best.pth.tar \
backend\pipeline\detectors\base.py:168: def run_predict(self, video_win_path: str,
output_win_dir: str,
backend\pipeline\detectors\base.py:179: def healthcheck(self) -> Tuple[bool, str]:
```

[Showing results with pagination = limit: 30]

29. assistant (2026-06-21T09:25:59.511Z)

I now have a complete picture. Let me verify the native runner's external dependencies (the container contents: WASB repo, weights, python_bin) to ground the Dockerfile claim, and confirm the worker's container entrypoint reality.

30. user (2026-06-21T09:26:00.862Z)

```
100 class NativeWasbRunner(DetectorRunner):
101     """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
102
103     def __init__(self, cfg: NativeWasbConfig):
104         self.cfg = cfg
105         # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
106         # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
107         self._cuda_available: Optional[bool] = None
108
109         # --- low-level native exec (the single place tests patch)
-----
110         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
111             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
112             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
113                                 timeout=(self.cfg.timeout_sec or None))
114
115         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
116         def _probe_cuda(self) -> bool:
117             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
118             (missing python / timeout / import error) is treated as 'no CUDA'."""
119             try:
120                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
121             except (FileNotFoundError, subprocess.TimeoutExpired):
122                 return False
123             return res.returncode == 0 and "cuda True" in (res.stdout or "")
124
125         def _cuda_is_available(self) -> bool:
126             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
127             if self._cuda_available is None:
128                 self._cuda_available = self._probe_cuda()
129             return self._cuda_available
130
131         def _effective_device(self) -> str:
132             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
133             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
134             if self.cfg.device == AUTO:
135                 return DeviceContext(AUTO, self._cuda_is_available()).effective
136             return self.cfg.device
137
138         def _repo_src(self) -> str:
139             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
```

```

140
141     def _copy_infer_script(self) -> bool:
142         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
143         configs/."""
144         repo_src = self._repo_src()
145         try:
146             os.makedirs(repo_src, exist_ok=True)
147             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
148             return True
149         except OSError as e:
150             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
151             return False
152
153     def _rm_rf(self, path: Optional[str]) -> None:
154         """Best-effort recursive delete of a native path (post-success cleanup; never
155         raises)."""
156         if path:
157             shutil.rmtree(path, ignore_errors=True)
158
159     def healthcheck(self) -> Tuple[bool, str]:
160         """Verify weights + repo present and the `wasb` python imports torch (and sees a
161         GPU on cuda)."""
162         if not self.cfg.weights_path:
163             return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
164             indexing.wasb.weights_path).")
165         weights = os.path.expanduser(self.cfg.weights_path)
166         if not os.path.exists(weights):
167             return False, f"WASB weights not found at {weights}"
168         repo_src = self._repo_src()
169         if not os.path.isdir(repo_src):
170             return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
171             nttcom/WASB-SBDT "
172             f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
173         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
174         # beats a cryptic
175         # argparse rc=2 from wasb_infer.py at run time.
176         if self.cfg.device not in VALID_DEVICES:
177             return False, (f"unknown device {self.cfg.device!r} ? valid: {'',
178             '.join(VALID_DEVICES)}").")
179         try:
180             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
181         except FileNotFoundError:
182             return False, f"python binary not found: {self.cfg.python_bin!r} (set
183             WASB_PYTHON).")
184         except subprocess.TimeoutExpired:
185             return False, "native torch healthcheck timed out."
186         out = (res.stdout or "").strip()
187         if res.returncode != 0:
188             return False, f"{self.cfg.python_bin}` torch import failed: {(res.stderr or
189             out).strip()}"
190         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
191         subprocess).
192         self._cuda_available = "cuda True" in out
193         ctx = DeviceContext(self.cfg.device, self._cuda_available)
194         if ctx.cuda_required_but_missing:
195             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
196             a silent
197             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
198             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
199         if ctx.fell_back:
200             logger.warning("device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO
201             CPU; WASB "
202             "inference will be slow. Set indexing.wasb.device=cuda to
203             require a GPU.", out)
204         return True, out

```

```

192
193         def run_predict(self, video_win_path: str, output_win_dir: str,
194                           video_id: Optional[str] = None) -> Optional[str]:

```

31. assistant (2026-06-21T09:26:17.366Z)

I now have everything I need. The native WASB runner depends on an external WASB-SBDT repo clone, weights file, and a `python\_bin` with torch+CUDA ? exactly the contents the M6 container image must bake in. Let me do a final check: confirm there is no existing Cloud Run dispatch test scaffold and confirm the worker is invocable as a container entrypoint (it is `python -m backend.worker infer`).

I have enough grounding. Let me compose the structured output for M6 and M7.

Summary of code-reality findings:

```

**Already exists (reusable verbatim for dispatch):**
- Async job control plane: `jobs` table (`database.py:187-203`, migration v2), `claim_due_job`
CAS (`database.py:588-608`),
`create_job`/`mark_job_done`/`mark_job_failed`/`set_job_progress`/`set_job_video_id`
(`database.py:486-536`).
- WAIT_AND_RESUME self-scheduling: `JobWaiting` (`job_worker.py:41-56`),
`set_job_waiting`/`seconds_until_next_due` (`database.py:559-586`),
`_drain_due_jobs`/`_schedule_next_drain_if_waiting`/`_arm_wake_timer` (`job_worker.py:100-152`).
The doc (D-line 76, §4.5) states this "carries to Cloud Run dispatch verbatim."
- Profile-agnostic worker CLI: `cmd_infer` pulls?runs?pushes via `storage.fetch`/`storage.put`
(`__main__.py:91-176`); `python -m backend.worker infer --video-uri ? --out-uri ?` is exactly
the container entrypoint.
- Storage substrate: `StorageBackend` ABC, `parse_uri`, GCS backend with
`download_to_filename`/`upload_from_filename`/`signed_url` v4 (`gcs.py:30-63`) ? covers bucket
I/O + reel-via-signed-URL for M7.
- Compute seam `_resolve_runner` (`trajectory_hybrid.py:66-87`) and `NativeWasbRunner` (cuda,
healthcheck on weights+repo+torch, `native_wasb_runner.py:157-191`) ? the in-container detector.
- Config seam: `compute_target` enum already has `"cloud_run"` (`models.py:351`); `cloud-
serving` preset (`presets.py:51-54`) sets `compute_target=cloud_run`, `detector_impl=native`.
These fields are inert in M1 (no behaviour rides on them yet).

```

```

**Net-new for M6:** a dispatch shim (when `compute_target=cloud_run`, instead of running in-
process, write job inputs to the bucket + trigger a Cloud Run GPU job via the
`run.googleapis.com` API, then `JobWaiting`-park the control-plane job and re-claim on
completion) + a `Dockerfile` baking ffmpeg + CUDA + WASB-SBDT repo + weights + fonts, with
`python -m backend.worker infer` as entrypoint. No `backend/compute/` dir, no Dockerfile, and
`deploy/gcp/` has only VM provisioning (not Cloud Run) exist today.

```

```

**Net-new for M7:** the actual L4 deploy + DEPLOY_REPORT ? pure owner spend/run, no code that
can be mock-tested in lieu of a real GPU.

```

32. user (2026-06-21T09:27:04.419Z)

Structured output provided successfully

33. assistant (2026-06-21T09:27:15.129Z)

I have mapped the M6 and M7 gates for the cloud-serving compute substrate, grounded in the code. The structured output is submitted.

Key findings:

```

**M6 (Cloud Run dispatch shim + container image)** ? the high-value, mostly-Claude-doable gate.
The async job control plane already exists and is reused verbatim: the `jobs` table,
`claim_due_job` CAS, the `JobWaiting`/WAIT_AND_RESUME park-and-reclaim loop, and a profile-
agnostic worker CLI (`python -m backend.worker infer --video-uri gs:/// --out-uri gs:///`) that
is already the exact container entrypoint. The `compute_target="cloud_run"` enum and the `cloud-
serving` preset already exist but are inert. Net-new is a `backend/compute/` ComputeBackend

```

registry (LocalCompute identity-default + CloudRunCompute behind `compute\_target=cloud\_run`, default-OFF) with an injectable Cloud Run client ? mirroring exactly the DI pattern
`GCSStorage(client=None)` already uses for testability ? plus a `Dockerfile` baking the deps
`NativeWasbRunner.healthcheck` asserts (ffmpeg, CUDA, WASB-SBDT repo, weights, fonts). All of it ships with fully-mocked dispatch tests + a GPU-free stub/cpu-mock image smoke for \$0. Owner prerequisite is just a design pick + go-ahead to write ? no spend.

****M7 (L4 e2e + DEPLOY_REPORT)**** ? pure owner spend/run. Almost nothing net-new is writable beyond M6; the load-bearing artifact is a real L4 run with real numbers and a contact sheet, which only the owner can produce (billing, GPU quota, golden corpus). I can pre-stage the DEPLOY_REPORT template and a mock-driven sample-run harness so the owner's first spend is a single deploy+invoke.

No code was modified; this was a read-only mapping pass.